

ASSIGNMENT – 01

Subject: Real Time Embedded System
(21EIC504J)

Title: QEMU for ARM Processor

Submitted by:

Bharathwaj Manoharan
(RA2512053010003)

1. Introduction: The Simulation Paradigm in Cyber-Physical Systems

The design methodology for modern embedded systems has undergone a fundamental paradigm shift, moving from hardware-dependent prototyping to model-based design and simulation. In the context of Cyber-Physical Systems (CPS), where computational elements control physical entities, the latency between software development and hardware availability presents a critical bottleneck. The Quick Emulator (QEMU) addresses this by providing a robust, machine-level emulation environment. For the postgraduate scholar, QEMU is not merely a utility but a study in architectural abstraction, allowing for the analysis of the ARM Instruction Set Architecture (ISA) and the validation of hardware-software interfaces (HSI) prior to silicon fabrication.

2. ARM Processor Architecture: The Guest System

To utilize QEMU effectively, one must analyze the target "Guest" architecture. The ARM architecture, governed by a RISC (Reduced Instruction Set Computer) philosophy, is stratified into profiles optimized for specific domains. QEMU must replicate the distinct behaviors of these profiles to ensure binary compatibility.

2.1 The ARM Cortex Profiles

The ARM ecosystem is segmented into three primary profiles. The emulation strategy for each differs significantly regarding interrupt latency and memory management.

- **Cortex-A (Application):** Characterized by the presence of a Memory Management Unit (MMU) supporting virtual memory. QEMU emulates the translation tables and Exception Levels (EL0-EL3) required for rich operating systems like Linux.
- **Cortex-R (Real-Time):** Focuses on deterministic behavior using a Memory Protection Unit (MPU) and Tightly Coupled Memory (TCM).
- **Cortex-M (Microcontroller):** The dominant architecture in IoT, defined by the Nested Vectored Interrupt Controller (NVIC). QEMU's fidelity in emulating the NVIC's tail-chaining and late-arrival mechanisms is critical for verifying real-time logic.

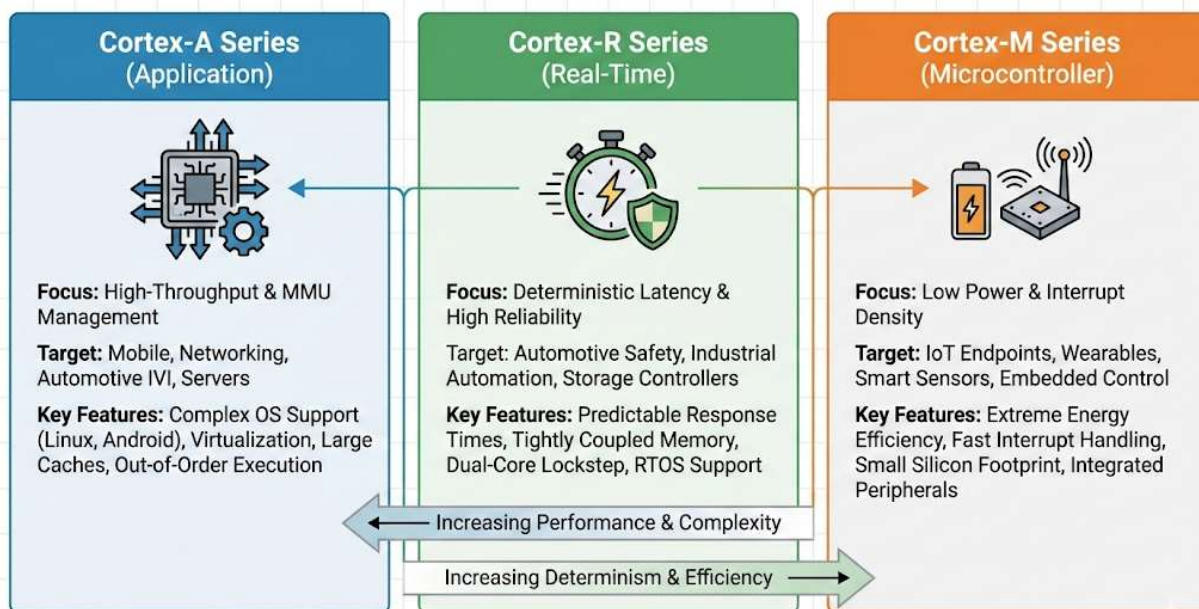


Figure 1: Architectural segmentation of the ARM Cortex families.

3. QEMU Architecture and Working Principle

QEMU operates on the principle of Dynamic Binary Translation (DBT). Unlike static simulators, QEMU translates guest code on-demand, caching the results to optimize future execution. The architecture is bifurcated into the Guest (ARM) and Host (typically x86_64) domains.

3.1 The Tiny Code Generator (TCG) Pipeline

The core engine facilitating this translation is the Tiny Code Generator (TCG). The translation process follows a rigorous three-stage pipeline:

1. **Frontend Translation (Guest to IR):** The vCPU reads ARM machine code (e.g., ADD R1, R2) and converts it into TCG Ops, a generic RISC-like Intermediate Representation.
2. **Optimization:** TCG performs liveness analysis and constant propagation on the IR to reduce instruction count.
3. **Backend Generation (IR to Host):** The backend maps IR ops to host-specific instructions (e.g., x86 ADD), handling register allocation and spilling.

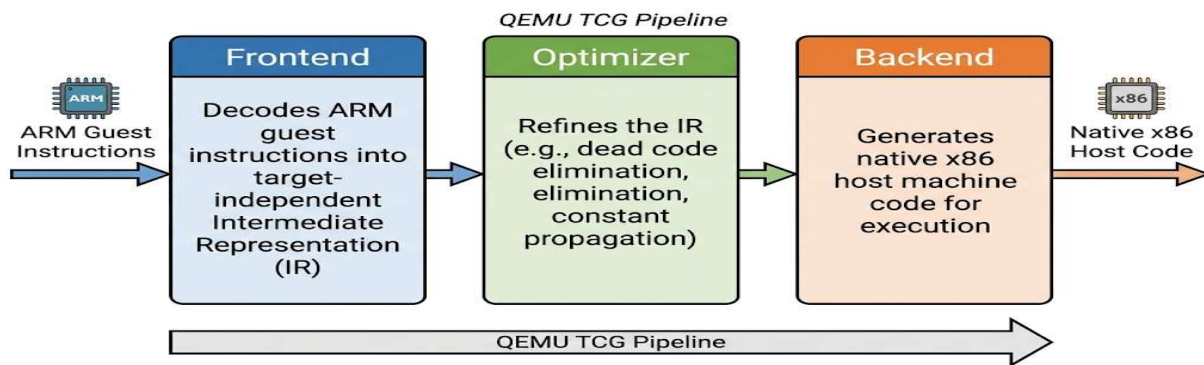


Figure 2: The QEMU Tiny Code Generator (TCG) Pipeline.

3.2 Block Chaining and the Translation Cache

To mitigate the overhead of the translation loop, QEMU employs Translation Blocks (TBs). A TB is a sequence of instructions ending in a branch. Once translated, TBs are stored in a `tb_jmp_cache`.

By patching the exit code of one TB to jump directly to the next, QEMU establishes a "Block Chain," allowing the host CPU to execute a stream of guest logic without context switching back to the emulator's main management loop.

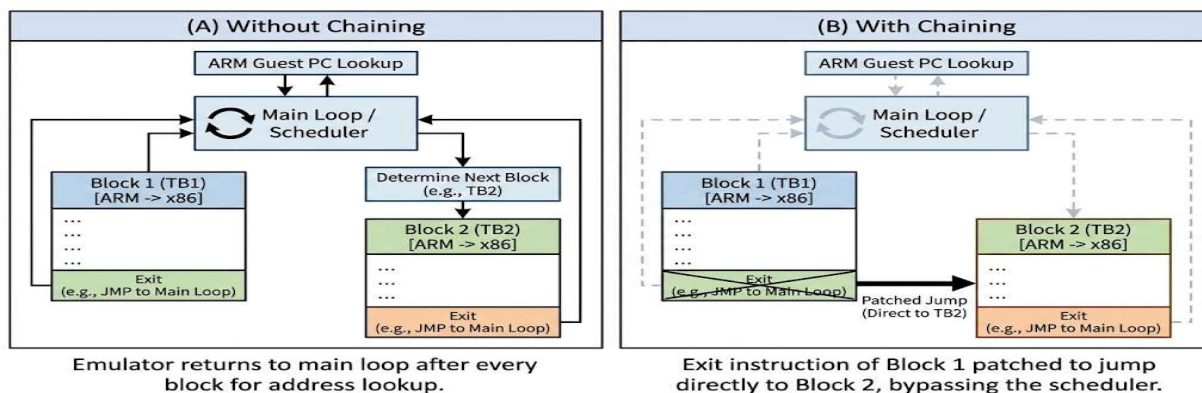


Figure 3: The Translation Block (TB) Chaining mechanism.

4. Relevance to Real-Time Embedded Systems

While QEMU runs on non-deterministic host operating systems (like Linux or Windows), it remains a vital tool for RTES development through specific mechanisms:

1. **Instruction Counting (icount):** QEMU can decouple the virtual clock from the host's wall-clock time, advancing it by a fixed number of nanoseconds per executed instruction. This enforces deterministic execution, enabling cycle-accurate replay debugging.
2. **Hardware-Software Co-Design:** It allows firmware validation to run concurrently with hardware development.
3. **Interrupt Logic Verification:** While precise timing verification requires physical silicon, QEMU validates the *logical correctness* of Interrupt Service Routines (ISRs) and priority nesting.

5. Emulation vs. Virtualization

In academic discourse, it is imperative to distinguish between emulation (TCG) and hardware-assisted virtualization (KVM/Xen).

- **Emulation (TCG):** Software-based translation. It is flexible (Cross-Architecture) but incurs performance overhead.
- **Virtualization (KVM):** Uses hardware extensions (Intel VT-x or ARM EL2) to execute guest code directly on the host CPU. It offers near-native performance but requires the guest and host architectures to match.

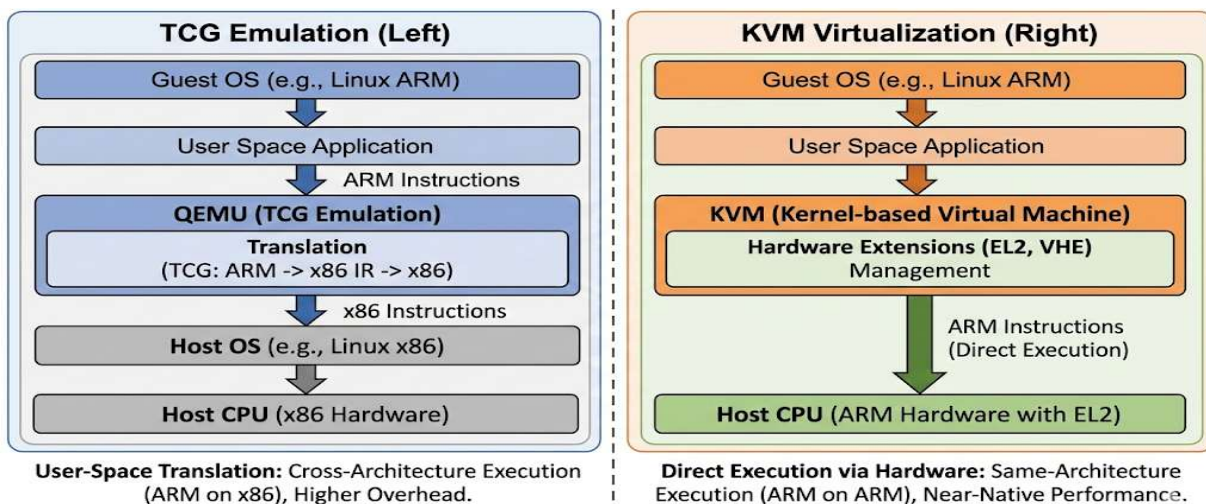


Figure 4: Comparative Stack of Emulation vs. Virtualization

6. Advantages and Limitations

6.1 Advantages

- **Introspection:** Complete visibility into register states and memory without intrusive hardware probes (JTAG).
- **Scalability:** Ability to spawn networked clusters of emulated devices for distributed systems testing.
- **Cost Efficiency:** Eliminates the need for physical evaluation boards for every student or developer.

6.2 Limitations

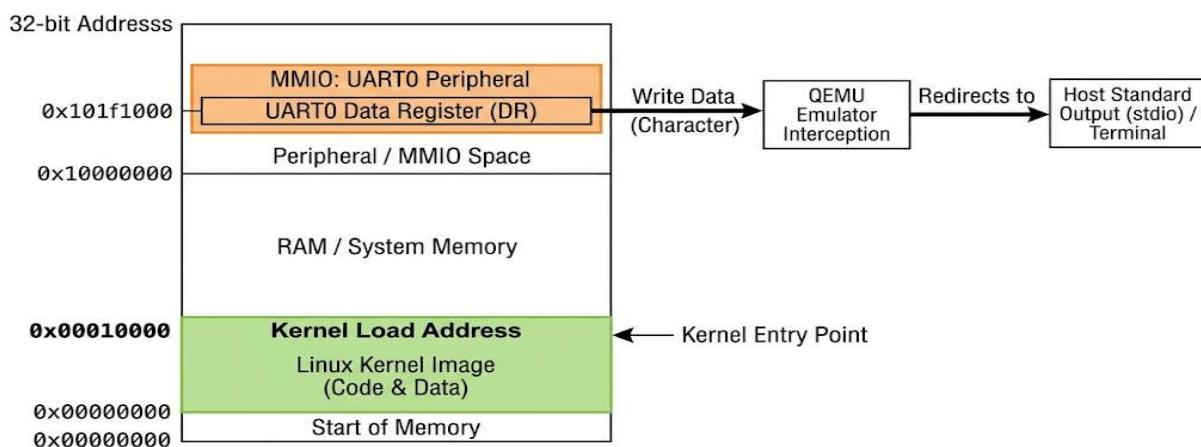
- **Timing Fidelity:** QEMU models functionality, not micro-architectural timing (e.g., pipeline stalls or cache contention). It is unsuitable for worst-case execution time (WCET) analysis.
- **Peripheral Accuracy:** Vendor-specific behaviors (e.g., complex power-gating sequences in specific SoCs) may be simplified or omitted in the device models.

7. Practical Implementation: Bare-Metal UART

To demonstrate the interaction between software and emulated hardware, we examine a bare-metal implementation on the VersatilePB machine model (ARM926EJ-S).

7.1 Memory Map Analysis

Figure 5: Schematic of the VersatilePB Memory Map.



Data written to UART0 DR at 0x101f1000 is intercepted by QEMU and redirected to the host's stdio.

The VersatilePB model maps the PL011 UART peripheral to a specific physical address. Writing to this address triggers the QEMU character device backend.

7.2 Implementation Code

The following C code defines a pointer to the hardware register. The volatile qualifier is essential to prevent the compiler from optimizing out the memory write, as the compiler is unaware of the side effects (transmission) caused by writing to this specific address.

```
File Edit Selection View Go Run Terminal Help
test.c - Visual Studio Code
C test.c > ...
1 #include <stdint.h>
2
3 /* Physical address of UART0 Data Register on VersatilePB */
4 volatile unsigned int * const UART0DR = (unsigned int *)0x101f1000;
5
6 void print_uart0(const char *s) {
7     while(*s != '\0') {
8         /*
9          * Write character to the hardware register.
10          * The hardware (or QEMU) consumes this data immediately.
11          */
12         *UART0DR = (unsigned int)(*s);
13         s++;
14     }
15 }
16
17 void c_entry() {
18     print_uart0("Hello, M.Tech Real-Time Systems!\n");
19     while(1); /* Halt execution */
20 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
1: bash
user@vscode-workspace:~/project$ arm-none-eabi-gcc -c test.c -o test.o
user@vscode-workspace:~/project$ arm-none-eabi-ld -T linker.ld test.o -o test.elf
user@vscode-workspace:~/project$ qemu-system-arm -M versatilepb -m 128M -nographic -kernel test.elf
Hello, M.Tech Real-Time Systems!
```

8. Advanced Emulation Workflows

Beyond simple peripheral access, QEMU enables advanced development workflows critical for modern embedded systems engineering.

8.1 Semi hosting: The Host-Target Bridge

While Memory Mapped I/O (MMIO) requires writing a driver for a specific UART peripheral, **Semi hosting** is a mechanism where the code running on the ARM target communicates directly with the QEMU host to perform I/O operations.

- **Mechanism:** The target executes a specific trap instruction (e.g., SVC 0x123456 on ARM). QEMU intercepts this exception, pauses the vCPU, and executes the system call (like printf or fopen) on the host machine using the host's standard libraries.
- **Application:** This allows developers to use standard C library functions (<stdio.h>) immediately after boot without initializing any drivers, significantly accelerating algorithm verification.
- **Command:** qemu-system-arm... -semihosting

8.2 Automated Testing and CI/CD Integration

In industrial RTES development, "Works on my machine" is insufficient. QEMU allows for **Headless Testing** within Continuous Integration (CI) pipelines (e.g., Jenkins, GitLab CI).

- **Workflow:**
 1. Code is committed to the repository.
 2. CI server cross-compiles the firmware.
 3. CI server launches QEMU in non-interactive mode with a timeout.
 4. Scripts parse the QEMU output (via stdout) to verify test cases (e.g., "TEST PASSED").
- **Significance:** This enables massive parallel testing of embedded firmware in the cloud, a capability impossible with physical hardware setups that require manual cabling.

8.3 TrustZone and Secure Boot Emulation

For security-critical research, QEMU supports the emulation of ARM TrustZone (EL3 Security Monitor).

- **EL3 Emulation:** QEMU can run Trusted Firmware-A (TF-A) and switch between the Secure World (Trusted OS) and Normal World (Linux/RTOS).
- **Research Value:** This allows M.Tech students to experiment with Secure Boot chains and cryptographic isolation without the risk of "bricking" expensive development boards.